AGILE SDLC DOCUMENT

LSTM-Based Sentiment and Emotion Analysis

Prepared from the supplied *LSTM.ipynb* notebook

Development Approach	Agile methodology with iterative development and sprint-based delivery
Project Type	Text classification using deep learning
Core Models	Stacked LSTM, Attention LSTM, Stacked CNN-BiLSTM, Hybrid CNN-BiLSTM
Core Technologies	Python, TensorFlow/Keras, Pandas, NumPy, Scikit-learn, Matplotlib, Seaborn, KaggleHub

Scope note: The notebook contains multiple dataset-specific experiments for sentiment and emotion classification. Dataset schemas, label mappings, class counts, sequence lengths, and training settings vary by experiment. This document summarizes the common workflow and identifies implementation details that should be validated before final reporting.

1. Introduction

This project analyzes text from social media and emotion datasets using deep learning. The notebook downloads datasets with KaggleHub, cleans text, encodes labels, tokenizes and pads sequences, splits data into training, validation, and test sets, trains several neural network architectures, and compares classification performance. Agile SDLC is suitable because data sources, preprocessing rules, model designs, and evaluation can be refined through short iterations.

2. Project Objectives

- Acquire text datasets from Kaggle using KaggleHub.
- Normalize and clean text by removing URLs, mentions, punctuation, numbers, and extra whitespace where configured.
- Encode sentiment or emotion categories and convert text into padded token sequences.
- Use a 70% training, 10% validation, and 20% test split.
- Train and compare multiple deep learning models.
- Evaluate classification using accuracy, weighted precision, weighted recall, weighted F1-score, and confusion matrices.
- Visualize model comparisons to support selection of a suitable architecture.

3. Agile SDLC Overview

Agile is an iterative and incremental approach. Work is organized into short sprints, each with a goal, prioritized tasks, a testable increment, and a review. Feedback from each sprint informs the next sprint. For this notebook, the proposed sprint plan is a practical project-management structure; it is not a claim that these sprints were historically completed.

Sprint	Goal	Increment / output
Sprint 1	Dataset acquisition and understanding	Downloaded dataset, inspected columns and label distributions
Sprint 2	Text preprocessing and encoding	Clean text, encoded labels, tokenized and padded sequences
Sprint 3	Baseline LSTM models	Trained initial Stacked LSTM and attention-based LSTM
Sprint 4	Model expansion	Trained CNN-BiLSTM and hybrid architecture variants
Sprint 5	Evaluation and visualization	Metrics table, confusion matrices, comparison charts
Sprint 6	Validation and refinement	Reproducible pipeline, checked splits, documented results and limitations

4. Product Backlog and User Stories

ID	User story / requirement	Acceptance criteria
US-01	As a developer, I want to download a supported dataset programmatically.	Dataset is downloaded and expected files/columns are available.
US-02	As a data analyst, I want consistent text cleaning.	Nulls and noisy text are handled and cleaned text is usable.
US-03	As a model developer, I want text converted into numeric sequences.	Tokenizer and padding produce arrays with the configured maximum length.
US-04	As a researcher, I want separate train, validation, and test sets.	The intended 70/10/20 proportions are checked and test data are held out.
US-05	As a researcher, I want to compare model architectures.	Models train and produce predictions on the same evaluation split.
US-06	As a reviewer, I want interpretable evaluation summaries.	Accuracy, precision, recall, F1, and confusion matrices are produced.

5. Sprint-wise Implementation

Sprint 1: Data Acquisition and Inspection

The notebook uses KaggleHub to retrieve datasets including a Twitter/Reddit sentiment dataset, airline tweets, tweet-emotion data, Twitter review emotion data, and an emotion classification dataset. Dataset-specific code reads the expected CSV or text files and selects the text and label columns.

Definition of done: Dataset download succeeds, file paths resolve, required columns are identified, and class distributions are inspected.

Sprint 2: Text Preprocessing and Dataset Preparation

Text-cleaning functions lowercase text, remove URLs and mentions where configured, remove non-alphabetic characters, and normalize whitespace. Labels are mapped to integers and converted to categorical arrays. Text is tokenized and padded to a fixed length. The notebook uses a 70/10/20 train/validation/test split.

Definition of done: No unexpected missing labels remain, sequences have consistent shapes, label encoding is valid, and split sizes are reported.

Sprint 3: Baseline Model Development

The notebook defines a Stacked LSTM and an Attention LSTM. The Attention LSTM uses an LSTM layer with sequence output, multi-head attention, pooling, and a softmax classification layer. Model training uses TensorFlow/Keras with the Adam optimizer and categorical cross-entropy in the described experiments.

Definition of done: Model builds successfully, training runs, validation metrics are captured, and prediction output dimensions match the number of classes.

Sprint 4: Additional Architectures

Additional models include a Stacked CNN-BiLSTM and a Hybrid CNN-BiLSTM. These architectures combine convolutional and recurrent layers, with pooling and dense classification layers as implemented in the notebook.

Definition of done: Each selected architecture trains without shape errors and uses the same evaluation protocol for a fair comparison.

Sprint 5: Evaluation and Visualization

The notebook computes accuracy, weighted precision, weighted recall, and weighted F1-score. It also creates confusion matrices and bar charts for comparing model performance.

Definition of done: Metrics are calculated from held-out test predictions, class ordering is consistent in confusion matrices, and figures are labeled.

Sprint 6: Review and Refinement

Review dataset-specific assumptions, remove duplicated or conflicting code where appropriate, make experiment settings explicit, and record reproducible outcomes. Confirm that no evaluation information leaks into model training.

Definition of done: The notebook can be rerun in a clean environment, dataset and model settings are documented, and reported results can be reproduced.

6. System Workflow

Stage	Description
1. Data source	KaggleHub downloads the selected sentiment or emotion dataset.
2. Preprocessing	Clean text, handle missing rows, and normalize input.
3. Label encoding	Map categories to numeric IDs and categorical targets.
4. Tokenization	Convert words to integer sequences and pad/truncate them.
5. Data split	Create training, validation, and test sets.
6. Model training	Train selected LSTM and CNN/BiLSTM variants.
7. Evaluation	Calculate metrics and confusion matrices on test predictions.

Stage	Description
8. Visualization	Compare model scores using charts.

7. Technology Stack

Technology	Purpose
Python	Main programming language.
KaggleHub	Programmatic dataset download.
Pandas / NumPy	Data manipulation and numerical operations.
TensorFlow / Keras	Neural network layers, model training, and prediction.
Scikit-learn	Dataset splitting and evaluation metrics.
Matplotlib / Seaborn	Metric charts and confusion-matrix visualization.

8. Testing Strategy

- **Data tests:** Verify expected files, columns, non-empty text, and valid labels.
- **Preprocessing tests:** Check URL/mention removal, whitespace normalization, and behavior on null or non-string input.
- **Sequence tests:** Verify tokenizer vocabulary limits, padding length, and consistent tensor dimensions.
- **Split tests:** Confirm train/validation/test proportions and ensure the test set is not used for model selection.
- **Model tests:** Check output class dimensions, finite loss, and successful inference.
- **Metric tests:** Verify metric inputs, class ordering, and confusion-matrix dimensions.
- **Reproducibility tests:** Set and record random seeds and package versions.

9. Risks and Mitigations

Risk	Impact	Mitigation
Dataset schema differs	Code may fail or labels may be misread.	Validate columns and label distributions for each dataset.
Class imbalance	Accuracy may hide poor minority-class performance.	Report weighted metrics and per-class results where useful.
Data leakage	Evaluation scores may be overly optimistic.	Fit preprocessing on training data where applicable and keep test data held out.
Inconsistent experiments	Model comparisons may be unfair.	Use identical splits and evaluation methods for models within an experiment.
Long training time	Slow iteration and repeated notebook runs.	Use controlled epochs, checkpoints, and documented experiment settings.
Multiple datasets and class mappings	Results may not be directly comparable across datasets.	Report dataset, label mapping, sequence length, and class count with each result.

10. Agile Roles and Ceremonies

For an individual or small academic project, roles may be combined. The following is a proposed structure:

- **Product owner:** Prioritizes goals such as robust preprocessing, model comparison, and clear evaluation.
- **Developer / data scientist:** Implements data processing, model architectures, training, and visualizations.
- **Reviewer / tester:** Checks reproducibility, data splits, metrics, and output correctness.

- **Sprint planning:** Select a small set of tasks and define acceptance criteria.
- **Daily check-in:** Record progress, blockers, and next actions.
- **Sprint review:** Demonstrate the increment, such as a cleaned dataset or model comparison chart.
- **Retrospective:** Identify what worked, what failed, and what to improve in the next iteration.

11. Limitations and Future Enhancements

- The notebook contains multiple dataset-specific experiments, so results from different datasets should not be combined into a single direct comparison without a consistent protocol.
- Some model settings, class counts, sequence lengths, and epoch counts differ between experiments.
- A tokenizer fitted on the full dataset before splitting can introduce information leakage from validation and test text; for a stricter pipeline, split first and fit the tokenizer on training text only.
- Combining original train/validation/test files into one dataset and randomly resplitting may change the original benchmark protocol; document this choice.
- Add stratified splitting where appropriate, per-class precision/recall/F1, learning curves, model checkpoints, and a saved label mapping/tokenizer.
- Add a simple interface for predicting sentiment or emotion from user-provided text.

12. Conclusion

Agile SDLC supports this project by organizing dataset acquisition, preprocessing, model development, evaluation, and refinement into short iterations. Each sprint delivers a testable result and enables improvements based on observed issues. The supplied notebook already contains multiple model architectures and evaluation visualizations; the next priority is to standardize experiment settings, prevent preprocessing leakage, and report reproducible test-set results with clear dataset context.

Appendix: Sprint Acceptance Checklist

- Dataset and source are recorded for each experiment.
- Text and label columns are verified before processing.
- Label mappings and class counts are saved.
- Train/validation/test sizes are checked.
- Tokenization and padding parameters are documented.
- Models are evaluated on held-out test data.
- Metrics and confusion matrices use consistent class order.
- Synthetic, benchmark, and independently collected data are clearly distinguished.
- Limitations and reproducibility instructions are included in the repository.